Introduction to Reinforcement Learning
5/10

Jean Martinet

MSc DSAI

2024 – 2025

Programme

- Introduction
  - Course 1 : Introduction to Reinforcement Learning (RL)
- Part I on tabular methods
  - Course 2 : Markov Decision Processes
  - Course 3 : Dynamic programming in RL
  - Course 4 : Temporal difference 1/2 (Q-learning)
  - **Course 5 : Temporal difference 2/2 (SARSA)**
- Part II on approximate methods
  - Course 6 : Value function approximation
  - Course 7 : Eligibility traces
  - Course 8 : Policy gradient 1/2 (REINFORCE)
  - Course 9 : Policy gradient 2/2 (actor-critic methods)
  - Course 10 : Projects presentation session

Reminder : think of a project topic

- **Choose from :**
  - Public presentation of articles/advanced topics/applications
    - Conference paper or book chapter
    - Advanced theme (e.g. actor-critic, eligibility trace, etc.)
    - Application domain (e.g. temperature control, revenue management, etc.)
  - Deepening or exploration project
    - Subject to be chosen/defined and validated
- **Choice to be validated... last week !**
- **Expected result :**
  - Short 2-page max PDF report
  - Code (ipynb / py / git)
  - Short 10-min presentation during last / before last session

About the project

- Double objective
  - Dig deeper in a specific subject (discussed or not during the lectures)
  - Share your insights with other students (in a teacher mode)
- A bit hard to choose early, before having reviewed all topics
- If you can define what is the environment, the reward, the agent, and the actions, it is a good start
- Stay small, at least for a first version, then make it more complex if you have time
- An experimental contribution is needed
  - E.g. compare two algorithms
  - E.g. start from an existing approach, and monitor changes when parameters vary
- The project needs be ORIGINAL
  - You need an original contribution of your own
  - Make sure your project is different from what can be found online
- IMPORTANT : if you decide to use an existing work, it is MANDATORY to cite the source, and you need to state what your contribution is

Today's menu

- Advantages of TD prediction methods
- SARSA
- Comparison with Q-learning through an example

Advantages of TD prediction methods

- No model of the environment is required (no P, no R)
- Online and incremental implementation
  - DP (and Monte Carlo) methods need to wait until the episode end
    - Only then the return is known
    - Besides, termination needs to be guaranteed
  - TD just waits until next step
  - TD methods update their estimates based on other estimates
    - They *bootstrap*

SARSA algorithm

- Previously : transitions from state to state, learn state values
- Now : consider state-action pairs, learn state-action pairs values
- SARSA is an on-policy TD algorithm

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

- The update rule uses every element of the quintuple $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$, which gives the name SARSA

SARSA algorithm

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\textit{terminal}, \cdot) = 0$

Loop for each episode:

  Initialize S

  Choose A from S using policy derived from Q (e.g., ε-greedy)

  Loop for each step of episode:

    Take action A, observe R, S'

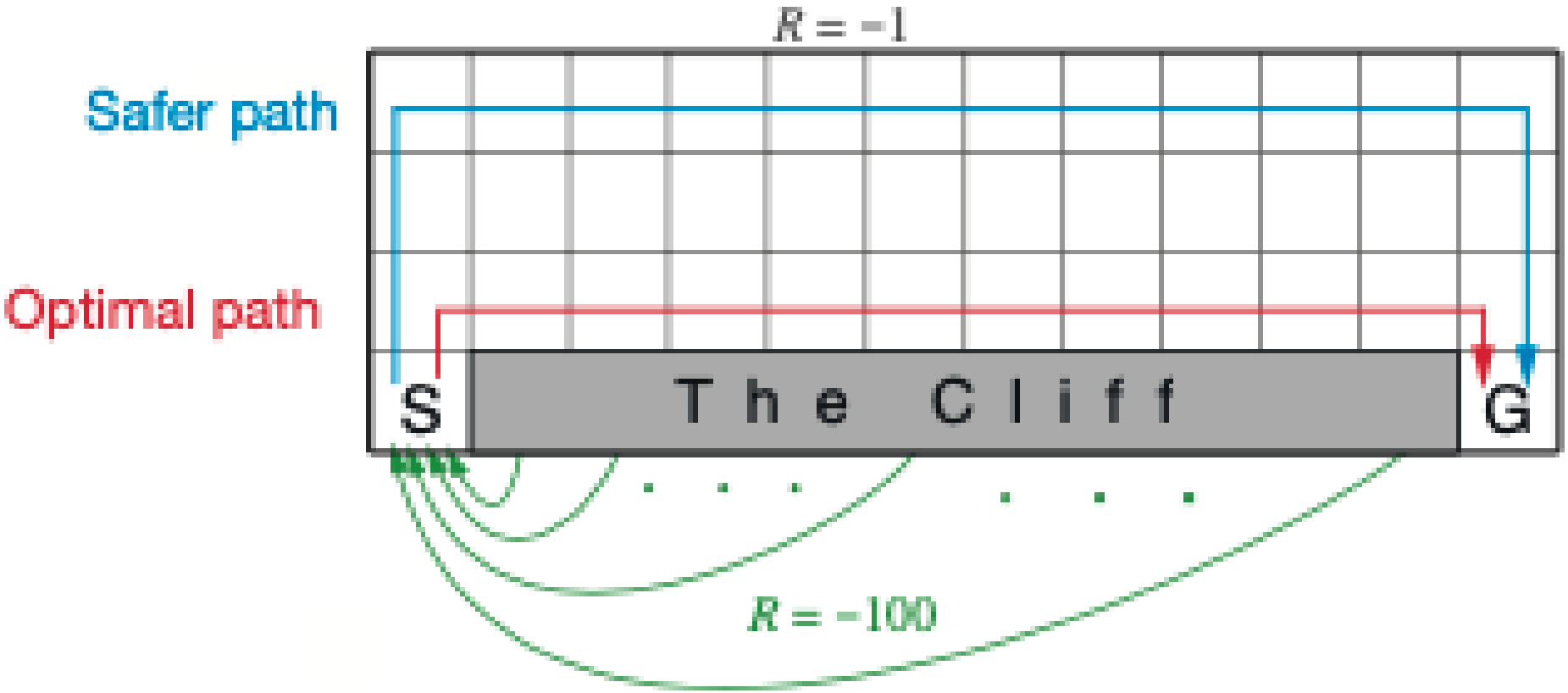
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)

    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$

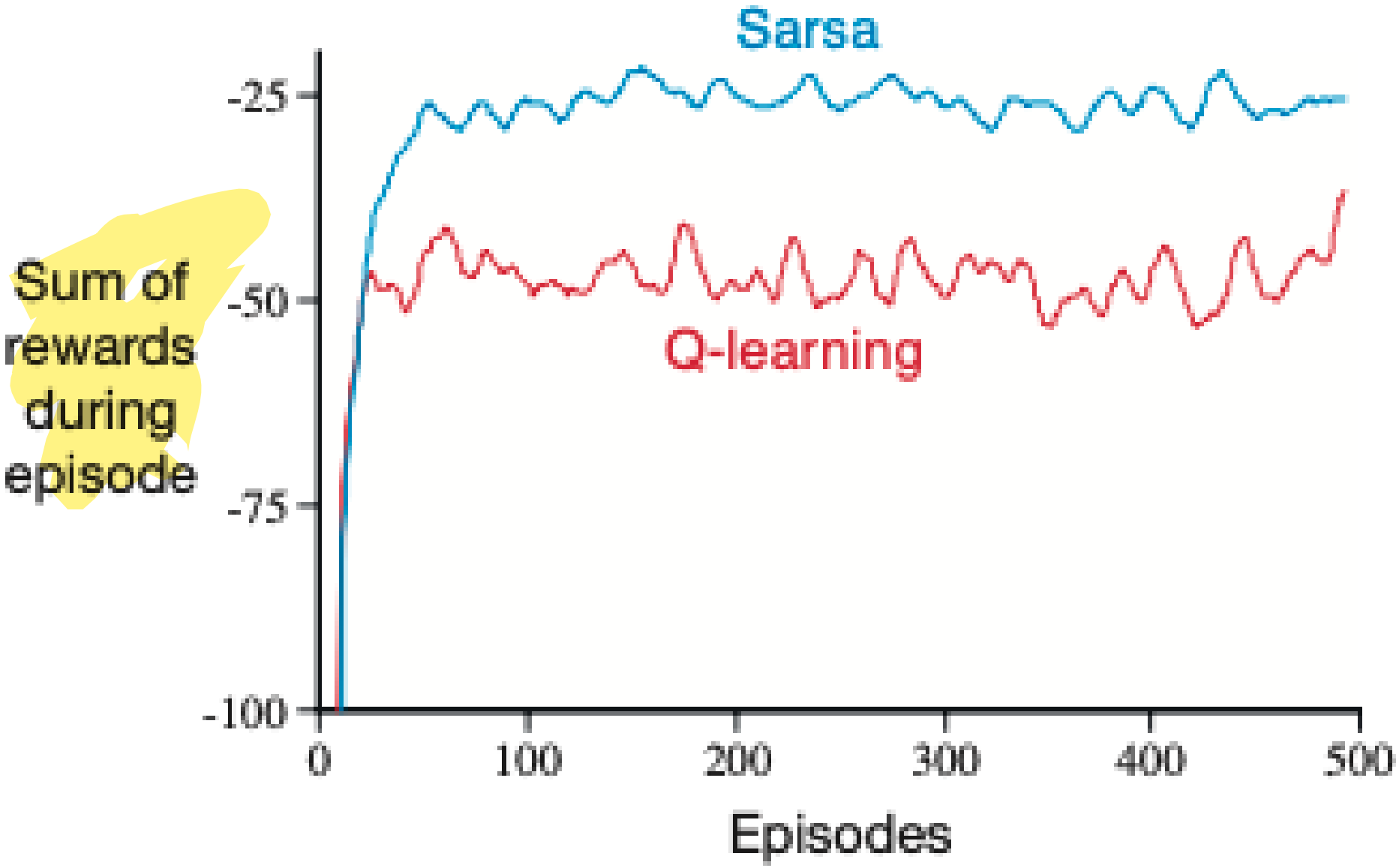
    $S \leftarrow S'$; $A \leftarrow A'$;

  until S is terminal

SARSA vs Q-learning : cliff walking example



SARSA vs Q-learning : cliff walking example



Today's lab

- Use gymnasium to implement and compare Q-learning and SARSA
  - See https://gymnasium.farama.org/environments/toy_text/cliff_walking/